Scaling Out Chip Interconnect Networks with Implicit Sequence Numbers

Giyong Jung
Dept. of Electrical and Computer Engineering,
Sungkyunkwan University
Suwon, South Korea
jyk2498@skku.edu

Saeid Gorgin
Dept. of Electrical and Computer Engineering,
Sungkyunkwan University
Suwon, South Korea
gorgin_81@yahoo.com

John Kim
School of Electrical Engineering,
KAIST
Daejeon, South Korea
jjk12@kaist.edu

Jungrae Kim
Dept. of Electrical and Computer Engineering,
Sungkyunkwan University
Suwon, South Korea
dale40@skku.edu

arXiv:2507.01988v1 [cs.NI] 28 Jun 2025

ABSTRACT

As AI models outpace the capabilities of single processors, interconnects across chips have become a critical enabler for scalable computing. These processors exchange massive amounts of data at cache-line granularity, prompting the adoption of new interconnect protocols like CXL, NVLink, and UALink, designed for high bandwidth and small payloads. However, the increasing transfer rates of these protocols heighten susceptibility to errors. While mechanisms like Cyclic Redundancy Check (CRC) and Forward Error Correction (FEC) are standard for reliable data transmission, scaling chip interconnects to multi-node configurations introduces new challenges, particularly in managing silently dropped flits in switching devices.

This paper introduces *Implicit Sequence Number (ISN)*, a novel mechanism that ensures precise flit drop detection and in-order delivery without adding header overhead. Additionally, we propose *Reliability Extended Link (RXL)*, an extension of CXL that incorporates ISN to support scalable, reliable multi-node interconnects while maintaining compatibility with the existing flit structure. By elevating CRC to a transport-layer mechanism for end-to-end data and sequence integrity, and relying on FEC for link-layer error correction and detection, RXL delivers robust reliability and scalability without compromising bandwidth efficiency.

KEYWORDS

Interconnect, Reliability, CXL, CRC, FEC, Implicit Sequence Number

1 INTRODUCTION

As Artificial Intelligence (AI) models continue to grow in size and complexity, their computational demands increasingly surpass the capabilities of individual processors [35, 41]. Training Large Language Models (LLMs) now requires extensive parallel processing across thousands of GPUs and CPUs [24, 28]. For instance, Meta trained the 405-billion-parameter Llama 3.1 model using 16K NVIDIA H100 GPUs and 4K CPUs [16]. Similarly, state-of-the-art AI inference tasks now depend on the combined computational power of multiple GPUs [5].

To support such distributed computing at scale, high-throughput interconnects between CPUs, GPUs, and accelerators have become essential. This demand has driven the development of advanced interconnect protocols, such as NVIDIA's NVLink [12], Compute Express Link (CXL)[7–9], and Ultra Accelerator Link (UALink)[30]. These protocols deliver exceptional bandwidth through advanced *physical layers* and operate on small data granularities (e.g., 64 bytes) at the *link layer* to reduce latency and minimize overfetching. Additionally, these protocols incorporate cache coherence at the *transaction layer*, allowing multiple chips to efficiently collaborate on shared data [23, 25].

Beyond protocol innovations, interconnect *infrastructures* have also evolved to enable scalable deployments. While traditional chip interconnects were initially designed for direct, point-to-point communication, modern systems now rely on scale-out architectures, where multiple processors communicate across shared networks. To facilitate this scalability, switching devices have become essential, enabling flexible scaling-out while preserving the high bandwidth of point-to-point links [53].

However, as transfer rates rise and interconnect networks scale out, concerns over interconnect reliability are intensifying. For instance, during the 54-day training of Llama 3.1, Meta reported seven job interruptions attributed to NCCL (NVIDIA Collective Communications Library) watchdog time-outs [16]. Failures in NVLink often manifest as stalled load/store operations, with the system triggering a time-out upon detecting these stalls. In a separate study, the Delta system [11] experienced a Mean Time Between Errors (MTBE) of only 6.9 hours for NVLink errors across 848 NVIDIA Ampere GPUs, with 66% of those errors resulting in job failures.

These incidents underscore the reliability challenges faced by modern interconnects. Furthermore, the problem is expected to worsen as signaling speeds escalate–from 50 GT/s in NVLink 3.0 (Delta) and 100 GT/s in NVLink 4.0 (Llama 3.1) to 200 GT/s in NVLink 5.0. Such aggressive scaling tightens timing margins and amplifies signal integrity issues, making transmission errors both more likely and more disruptive, especially in large-scale deployments.

'SC 2025', Nov 16–21, 2025, St. Louis, USA
2025. ACM ISBN 979-8-3503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

To mitigate the rising physical-layer error rates, modern protocols incorporate robust link-layer error recovery mechanisms. For example, CXL 3.0 doubles the transfer rate of its predecessor to 64 GT/s by relaxing the acceptable Bit Error Rate (BER) to 10^{-6}–six orders of magnitude higher than CXL 2.0's 10^{-12}. To maintain reliable communication, CXL 3.0 introduces an 8-byte Cyclic Redundancy Check (CRC) and 6-byte Forward Error Correction (FEC) per 256-byte flit [8, 9]. CRC detects flit corruption for retransmission, while FEC corrects many errors on-the-fly, reducing the reliance on retransmission and ensuring robust performance.

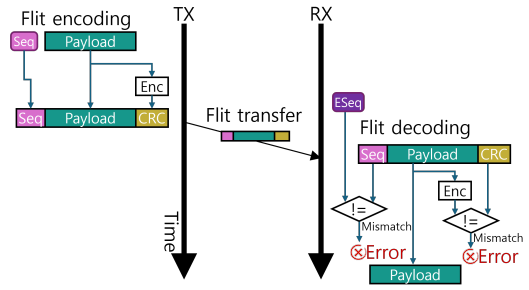
While CRC and FEC provide effective protection in direct links, these mechanisms fall short in more complex, switched environments. In direct connections, CRC and FEC rely on the flit successfully reaching its destination to trigger error recovery [34]. However, in switched environments, intermediate devices often discard erroneous flits without notifying the final destination [15, 18, 29, 36, 38], leading to silent flit drops. These drops disrupt sequence alignment and data integrity at the endpoint, particularly in systems lacking mechanisms to track dropped flits [37].

Traditional networking protocols, such as the *Transmission Control Protocol (TCP)*, address such challenges by incorporating two dedicated fields in their headers for sequence tracking: the *sequence number (SeqNum)*, which indicates the order of the current packet, and the *acknowledgment number (AckNum)*, which represents the last correctly received packet in the reverse direction (i.e., via ACK piggybacking) [49]. Together, these fields allow the sender and receiver to detect and recover from missing or out-of-order packets, ensuring reliable and ordered communication even in complex network environments.

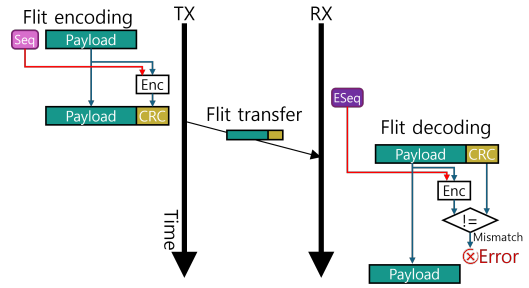
However, this approach introduces substantial header overhead, such as the 4-byte SeqNum and 4-byte AckNum fields in TCP. While such overhead is manageable for networking payloads, which are often a few kilobytes in size, it is impractical for chip interconnect protocols with much smaller payloads (e.g., 64 bytes), where minimizing header size is critical. For example, CXL's 256-byte flits dedicate only a two-byte header to sequence tracking and other control fields.

As a result, CXL's 256-byte flits allocate a single 10-bit field for sequence tracking, which serves dual purposes as either a SeqNum or an AckNum [9]. While this approach minimizes header overhead, it introduces inherent limitations. A single field cannot simultaneously manage sequencing and acknowledgment effectively. For example, when a flit uses the field to carry an AckNum, the receiver loses visibility of the flit's SeqNum, making it impossible to confirm whether the flit is part of the expected sequence (Section 4.1). This lack of sequence tracking creates critical vulnerabilities in multi-node environments where switching devices are prone to dropping flits (Section 7.1).

This paper introduces a novel concept called *Implicit Sequence Number (ISN)*, designed to address the challenges of achieving accurate sequence tracking while maintaining minimal flitization overhead. ISN eliminates the need for explicitly transmitting a SeqNum in flit headers by implicitly embedding the sequence information within the CRC checksum. This approach leverages the inherent redundancy in error recovery mechanisms, as both CRC violations and SeqNum mismatches ultimately trigger the same response: *retry*.



(a) An example of flit processing with explicit sequence numbers. The flit header includes a sequence number.



(b) An example of flit processing with *implicit sequence numbers*. The header has no sequence number but the CRC is generated based on both the payload *and* sequence number.

Figure 1: Comparison between explicit sequence numbers (used in prior work) and implicit sequence numbers (ISN).

Fig. 1 illustrates the difference between traditional explicit sequence numbers and ISN. In ISN, the CRC is encoded using both the payload and the SeqNum, effectively embedding the sequence number into the CRC checksum itself. At the receiver, CRC decoding incorporates the received payload and the *expected sequence number (ESeqNum)*. Any mismatch–either in the payload or between SeqNum and ESeqNum–triggers a CRC error, enabling the detection of missing flits through CRC results alone.

By eliminating the need for explicit sequence number fields, ISN offers a lightweight and practical solution that is compatible with a broad range of interconnect protocols. Many open standards, such as PCIe [34] and UCIe [50], already employ compact headers in which a single field is multiplexed as either a SeqNum or AckNum, thereby inheriting similar limitations. Although proprietary protocols like NVLink provide limited public documentation, they are likely to adopt comparable strategies for minimizing header overhead. In this work, we apply ISN to the CXL protocol and demonstrate how implicit sequence tracking can substantially improve reliability—particularly in scale-out architectures that rely on switching devices.

The enhanced protocol, referred to as *RXL (Reliability Extended Link)*, integrates ISN into the transport layer of CXL, extending reliability across the entire interconnect network. In CXL, CRC and FEC are confined to the link layer, providing error detection and correction for individual links. In contrast, RXL shifts the CRC functionality to the transport layer, enabling end-to-end protection for both data integrity and sequence integrity. RXL generates the CRC using SeqNum at the originator and decoded at the final

destination using the ESeqNum. This shift eliminates the need for switching devices to manage sequence numbers for the modified CRC decoding and enables end-to-end data protection against link-level errors and internal errors within switches. Meanwhile, FEC continues to operate at the link layer in RXL, providing robust error correction and detection to minimize unnecessary transfers of erroneous flits. This dual-layer approach ensures reliable communication across multi-node systems, making RXL a scalable and efficient interconnect solution for modern distributed computing environments.

The major contributions of this paper are as follows:

- We identify critical limitations of current chip interconnect protocols in handling silently dropped flits, which hinder its ability to scale out effectively. To the best of our knowledge, this is the first work to address protocol-level reliability challenges in chip interconnects.
- We propose *Implicit Sequence Number (ISN)*, a novel mechanism that integrates sequence tracking into the CRC, eliminating the need for explicit sequence numbers in headers.
- We introduce *RXL*, an enhancement of the CXL protocol that implements ISN. RXL enables robust detection of sequence misalignments and silent flit drops while maintaining compatibility with existing flit structures.

2 BACKGROUND

2.1 Chip Interconnect Errors

Errors on chip interconnects have historically been underexplored by the computer architecture community. However, field studies have suggested that interconnect reliability can become a critical bottleneck in large-scale systems [14, 16, 17]. For instance, a study on the Jaguar XT5 supercomputer [17] reported that approximately 10% of application interruptions were related to its SeaStar2+ communication processors [3]. Similarly, the IBM Mira supercomputer has reported that 21% of fatal events are related to its Blue Gene/Q link modules [14].

More recently, Meta conducted a root-cause analysis of job interruptions during the 54-day training of its Llama 3.1 model with 405B parameters [16]. This system utilized 16K NVIDIA H100 GPUs, with eight GPUs per server interconnected via NVLink 4.0 (100 GT/s). Among 322 job interruptions caused by hardware failures, 7 cases (2.2%) were attributed to NCCL watchdog timeouts, a potential indicator of NVLink issues.

A separate field study on the Delta system [11] observed that 4.7% of GPU errors were due to NVLink errors (XID=74) in the 206-node system comprising 848 NVIDIA Ampere GPUs. These errors translated to a MTBE of 1,415 node hours or just 6.9 hours at the system level. Alarmingly, 66% of the NVLink errors led to job failures, and 14% affected two or more GPUs, despite the use of mitigation strategies such as memory error containment. In terms of job interruptions, NVLink errors accounted for 3.7% of all cases. However, after excluding a small number of faulty HBMs responsible for 90% of uncontained memory errors, the contribution of NVLink errors rose significantly—accounting for 10.2% of the remaining job interruptions. This finding underscores the growing impact of interconnect errors in modern GPU clusters, particularly as other sources of failure become better contained.

These reliability concerns are expected to worsen as interconnect bandwidth continues to scale aggressively. For instance, CXL 3.0, operating at 64 GT/s, has relaxed its BER tolerance to 10^{-6}. Similarly, the UALink protocol [10], which is based on the physical layer of 200 GT/s Ethernet (IEEE 802.3dj), could experience a BER as high as 2.4×10^{-4}[2]. BER requirements of proprietary solutions like NVIDIA's NVLink 5.0 (200 GT/s)[32] and Google's ICI (Inter Chip Interconnect, 400 GT/s) [33] are not publicly available, but they are highly likely to face comparable or even greater error vulnerabilities due to their aggressive data rates. These trends underline the urgency of addressing interconnect reliability in future large-scale systems.

2.2 Compute Express Link (CXL)

CXL is an open-standard, cache-coherent interconnect designed to enable low-latency, high-bandwidth communication among CPUs, memory, accelerators, and peripheral devices. Built on top of the PCIe *physical layer*, CXL introduces three distinct protocols at the *link* and *transaction* layers: `CXL.io`, `CXL.cache`, and `CXL.mem` [9]. While CXL.io maintains compatibility with PCIe for basic I/O operations, CXL.cache and CXL.mem enable low-latency, high-bandwidth memory access with cache coherence. According to a recent study, CXL can deliver up to 5.7× higher data throughput in multi-node systems compared to PCIe [47], underscoring its advantage in supporting data-intensive applications in data centers [44].

CXL achieves high bandwidth through advanced physical layer techniques. CXL 3.0 employs PAM4 (Pulse Amplitude Modulation with four levels), which transmits 2 bits per symbol at a signaling rate of 64 GT/s [19, 43]. However, this modulation imposes a tight 31ps timing window per symbol, making signal integrity a significant challenge. To mitigate inter-symbol interference (ISI), CXL incorporates Decision Feedback Equalization (DFE), which adjusts signal levels based on previously received symbols [4]. While DFE effectively mitigates ISI, an incorrect decision can propagate errors across subsequent symbols [31].

Reflecting these challenges, CXL 3.0 relaxed its bit error rate (BER) tolerance to 10^{-6}–a sharp increase from CXL 2.0's 10^{-12}. As a result, a full-speed ×16 CXL 3.0 link (1 Tb/s) may experience up to one million bit errors per second. Furthermore, these *first bit errors* can further propagate through the DFE, manifesting as burst errors at the link layer. These conditions necessitate robust error detection and correction mechanisms above the physical layer to ensure end-to-end reliability.

The CXL link layer ensures reliable data transfer by managing flow control and error handling. To accommodate high data rates and small block sizes (e.g., transferring 256 bytes over a ×16 CXL 3.0 link takes just 2 ns), data is transmitted in fixed-size flow control units (flits) rather than variable-size packets. CXL 3.0 defines two flit sizes: 68B and 256B [9]. In the full-speed mode (64 GT/s), 256B flits are employed to improve reliability with additional headers. At reduced speeds, 68B flits can be used to minimize latency and reduce overfetching.

At the transaction layer, CXL supports cache-coherent access between hosts and devices, based on the MESI (Modified, Exclusive, Shared, Invalid) protocol [13]. A transaction involves three *messages*: a request, a response, and data. For example, a device may

<table>
<tr><td colspan="3">Source port (16-bit)</td><td>Destination port (16-bit)</td></tr>
<tr><td colspan="4">Sequence number (32-bit)</td></tr>
<tr><td colspan="4">Acknowledgement number (32-bit)</td></tr>
<tr><td>Offset</td><td>-</td><td>Flags (9-bit)</td><td>Window size (16-bit)</td></tr>
<tr><td colspan="3">ECRC Checksum (16-bit)</td><td>Urgent pointer (16-bit)</td></tr>
</table>

Figure 2: The TCP header format (20B).

issue a RdCurr request to retrieve the latest cache line version from the host. The host processes this request, updates its cache hierarchy, sends a response, and then transfers the requested data. To improve link efficiency, the link layer can pack tens of transaction messages into a single 256B flit.

2.3 CXL Flit Errors

Bit errors in CXL transmissions manifest as *flit corruption* or *flit drops*. Flit corruption occurs when an erroneous flit reaches the endpoint. At the endpoint, the error can be detected using CRC, and a retry is initiated to recover the corrupted data.

In contrast, flit drops occur when an erroneous flit is discarded by an intermediate switching device. The CXL standard does not explicitly define how switches should handle corrupted flits, and no publicly available data sheets provide clarification. However, datasheets for existing PCIe and Ethernet switches [15, 18, 29, 36, 38] suggest that switching devices typically discard packets upon detecting errors, which is practical because forwarding corrupted flits risks misrouting caused by undetectable address corruption [45].

Flit corruption and drops pose significant threats to system reliability. These errors can propagate to higher layers, jeopardizing computation accuracy and overall system stability. Cache-coherent protocols are particularly vulnerable because they rely on the strict ordering of requests, responses, and data to maintain consistency. Any disruption caused by flit drops can lead to cache inconsistencies, erroneous data sharing, and unpredictable behavior.

The risks are amplified by CXL's ability to pack multiple transaction messages into a single flit. For example, a 256B flit can carry up to 44 messages (request, data-header, or response) per 128B group. Losing such a flit could disrupt all associated transactions, amplifying the impact on system stability and data integrity.

2.4 TCP Reliability Mechanisms

CXL relies on its link layer alone for reliable transmission over individual links. In contrast, scaled-out network protocol stacks like TCP/IP incorporate a transport layer on top of a link layer to ensure accurate, ordered, and end-to-end data delivery across networks [39]. This section reviews the reliability mechanisms of TCP, a transport layer protocol designed to handle the diverse and error-prone nature of the Internet. RXL adapts some of these ideas to enhance reliability in scaled-out interconnect networks while preserving the existing CXL flit structure.

The TCP header format (Fig. 2) includes three key fields for reliability: the 32-bit *sequence number (SeqNum)*, the 32-bit *acknowledgment number (AckNum)*, and the 16-bit *End-to-End CRC (ECRC)*.

2.4.1 Sequence Number. This field ensures in-order delivery of data by uniquely identifying the position of each segment in the data stream. During connection setup, the sender and receiver exchange 32-bit initial sequence numbers to synchronize communication. Then the sender includes the *sequence number* in the TCP header of each transmitted segment and increments it for subsequent segments. The receiver tracks an *expected sequence number*, which allows it to detect missing or out-of-order segments, enabling detection of packet drops and reassembly of the data stream.

2.4.2 Acknowledgment Number. Acknowledgment numbers complement sequence numbers by providing feedback on received segments. If a segment is missing or fails CRC validation, the receiver sends a NACK (Negative ACKnowledgment) containing the last correctly received sequence number, prompting the sender to retransmit the missing data. Upon successful receipt, the receiver sends an ACK (ACKnowledgment) to free the sender's buffer for acknowledged segments.

ACK messages can generate significant traffic in the reverse direction. To reduce the reverse-direction traffic, TCP employs two mechanisms: *ACK coalescing* and *ACK piggybacking*. Coalesced ACKs aggregate acknowledgments to reduce the frequency of individual ACKs, while piggybacking attaches an acknowledgment to the next outgoing data packet. To support piggybacking, TCP headers include an additional sequence number: a 32-bit acknowledgment number (AckNum), allowing both data transmission and acknowledgment in a single packet.

2.4.3 End-to-End CRC. Although the link layer utilizes CRC to allow reliable transmission within a link, errors outside individual links (e.g., within switching devices) are not protected by the *Link CRC (LCRC)*. To fill this reliability gap, TCP utilizes a 16-bit checksum as End-to-end CRC (ECRC). The sender generates the checksum and embeds it in the segment header. At the endpoint, the receiver recalculates the checksum and compares it to the transmitted value to ensure end-to-end data integrity.

While TCP's mechanisms provide robust reliability, they incur significant header overhead. For example, the TCP header size is 20B, and when combined with IPv6 (40B) and Ethernet (14B) headers, the total overhead reaches 74B. This overhead is typically amortized by large payloads (e.g., 1 KB) of network traffic. However, it is less suited for small data units like those used in chip-to-chip interconnects.

2.5 Shortened Reed Solomon Codes

Wired communication systems often employ Reed-Solomon (RS) codes for FEC due to their robust and efficient error-correction capability [27]. These codes can correct up to t symbol errors within a codeword by utilizing $2t$ redundant symbols. The maximum codeword size is $2^m - 1$ symbols, where m is the size of each symbol in bits. For instance, with 8-bit symbols, RS codes can provide *Single Symbol Correction (SSC)* for a 255-symbol codeword using just 2 redundant symbols [6].

In practical applications such as CXL, however, the data block sizes are often smaller than the maximum codeword size. To accommodate this, RS codes are typically *shortened*. During the encoding process, unused symbol positions in the codeword are padded with zeroes to match the actual data block size. At the decoding stage, any correction attempt targeting these constant zero-padded positions is flagged as invalid, enabling the decoder to detect some errors beyond the correction threshold.

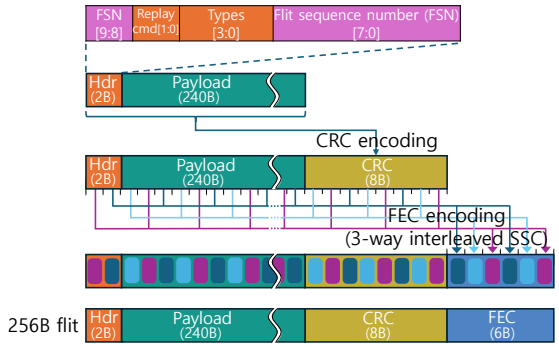


Figure 3: The structure and encoding of CXL 256B flits.

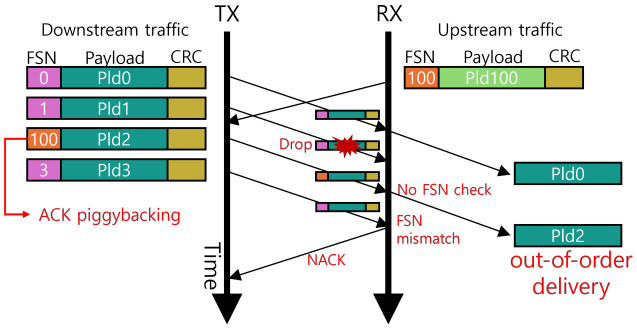


Figure 4: An example of CXL's failure on handling flit drops at the link layer.

This extra detection capability allows FEC to function as a robust first line of defense against transmission errors. In CXL, a 256B flit is divided into three sub-blocks of 85, 85, and 86 bytes. Each sub-block is protected by 8-bit symbol RS codes, which provide SSC with 2 redundancy bytes, as shown in Fig. 3. These shortened codes can detect approximately two-thirds of uncorrectable errors due to the presence of 170 unused positions in the 255-symbol codeword. As a result, the FEC with 3-way interleaved SSC can correct up to 3-symbol burst errors and detect 2/3 of 4-symbol burst errors, 8/9 of 5-symbol burst errors, and 26/27 of 6-symbol burst errors or larger, as it requires all three sub-blocks to miscorrect for such burst errors to go undetected.

RXL leverages these detection capabilities of FEC to restructure the CXL protocol stack. It exclusively relies on FEC for local error detection and correction at the link layer, while elevating CRC to the transport layer for end-to-end validation of both data and sequence integrity at the endpoints (Section 6).

3 PRIORWORK

The concept of leveraging data protection mechanisms to encode auxiliary metadata has been explored in several prior works, including Implicit-Storing [40], All-Inclusive ECC [22], and Implicit Memory Tagging [48]. These approaches utilize ECC redundancy to protect address signals or to embed metadata such as memory tags without requiring additional storage overhead.

In contrast, this work targets the unique reliability challenges of high-speed, multi-node chip interconnects. Specifically, it addresses sequence ordering violations caused by flit drops in scaled-out interconnects, a critical issue overlooked in current protocols. By embedding sequence tracking into CRC via ISN, this approach ensures robust sequence validation and end-to-end data integrity for chip interconnects, offering a scalable solution to the growing challenges posed by error-prone interconnect technologies.

Recent works on memory ECC have leveraged the underutilized syndromes of shortened RS codes to enhance their correction capabilities [20, 21, 26, 46]. In contrast, ISN takes advantage of the partial detection capabilities of shortened RS codes to enable early error detection based on FEC.

4 MOTIVATION

This section examines the reliability mechanisms employed in CXL 3.0 and their weakness. The discussion primarily centers on the 256B flit structure, as 68B flits are limited to lower-speed modes and are unsuitable for high-performance configurations.

4.1 Analysis of CXL Reliability

CXL 3.0 incorporates several reliability mechanisms to handle its high frequency of bit errors (up to 10^{-6} BER). Each 256B flit comprises a 2B header, a 240B payload, an 8B CRC, and a 6B FEC (Fig. 3). The 8B CRC is generated from the 2B header and 240B payload, providing robust detection capabilities. It can detect up to four random bit errors and burst errors up to 64 bits long with complete reliability. For more severe errors, it achieves an extremely high detection probability of $1 - 2^{-64}$. This ensures that even rare multi-bit errors are identified with near certainty.

The FEC splits the 250B combined header, payload, and CRC into 83B, 83B, and 84B sub-blocks and appends 2B redundancy to each sub-block using 8-bit symbol RS codes. These redundant symbols enable SSC within each sub-block, while the 3-way interleaving mechanism allows the FEC to correct up to three-symbol burst errors. According to the PCIe 6.0 standard, the flit error rate after FEC must not exceed 3×10^{-5} [34].

This combination of CRC and FEC provides robust error detection and correction, significantly enhancing data integrity. Uncorrectable flits—those with errors spanning multiple symbols within a sub-block—are detected with moderate probability by the FEC (Section 2.5) and with exceedingly high probability by the CRC, which boasts an undetected flit error rate below 1.6×10^{-24} (Section 7.1). To achieve this robustness, however, CXL 3.0 allocates 5.5% of the flit size (14 bytes) to redundancy.

While CRC and FEC provide robust protection against data corruption, they are ineffective against silent flit drops, as their operations are confined to received data. To address this limitation, CXL 3.0 incorporates a 10-bit *Flit Sequence Number (FSN)* and a 2-bit *ReplayCmd* field within the 2B header [9] (Fig. 3). The FSN serves multiple purposes based on the ReplayCmd setting:

- *ReplayCmd* = 0: The FSN contains the explicit sequence number of the current flit.
- *ReplayCmd* = 1: The FSN carries the acknowledgment sequence number (AckNum).
- *ReplayCmd* = 2: The FSN specifies the last valid received SeqNum for NACK with go-back-N retry.
- *ReplayCmd* = 3: The FSN specifies the last valid received SeqNum for NACK for single flit retry.

While this multi-purpose FSN design reduces header size, it introduces reliability trade-offs. Fig. 4 demonstrates a potential failure scenario in CXL's handling of flit drops. In this example, a host transmits downstream flits numbered #0, #1, #2, and #3, while the device sends an upstream flit numbered #100. To acknowledge the upstream flit, flit #2 carries an AckNum (100) in its FSN field instead of its own sequence number (2).

Suppose flit #1 is dropped due to an error. The device should ideally halt the processing of subsequent flits to maintain the ordering. However, because flit #2 does not include its own SeqNum, the device can only conduct data integrity checks (using CRC and FEC) but cannot verify sequence integrity. Upon passing the data integrity check, the device forwards flit #2 to application layers, despite the missing flit #1.

Later, when flit #3 arrives, the device can detect the missing flit by comparing the SeqNum (3) with the ESeqNum (2). However, by this time, the system has already forwarded the incorrectly ordered flit sequence to the application layer, rendering the late detection ineffective. This premature forwarding can potentially disrupt a large number of messages packed within the missing flit (up to 44 messages per 128B group), which could lead to unpredictable behaviors and inconsistencies across caches (Section 4.2).

This reliability hole due to incomplete sequence tracking can be exacerbated in multi-level switching environments. Although the switches can correct most erroneous flits using FEC, the remaining flits can result in intermediate flit drops. As the number of switching levels increases, these uncorrectable flit drops can accumulate, causing a proportional rise in flit drops at the endpoint.

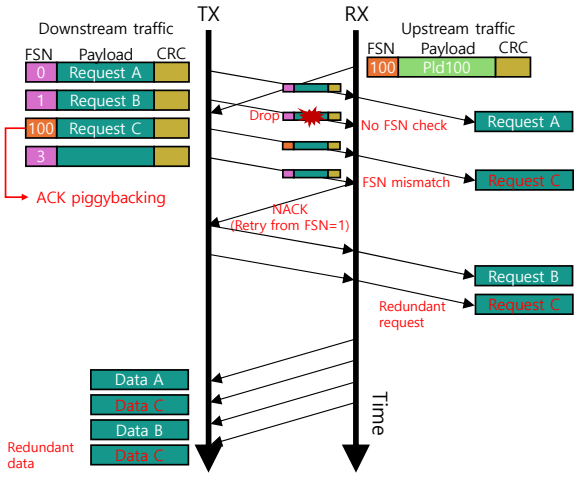
4.2 Impact of CXL Vulnerability

The reliability gap at the link layer can lead to severe consequences at higher layers, including the transaction and application layers.

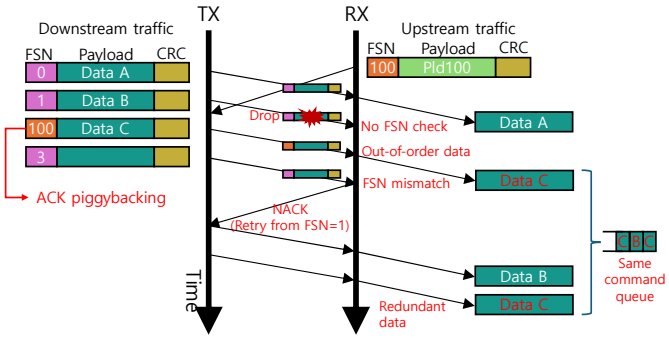
Fig. 5a illustrates an example scenario involving request-carrying flits. In this example, flit #1 (the dropped flit) and flit #2 (the misforwarded flit) carry read requests B and C, respectively. Due to the silent drop of flit #1 and the absence of proper sequence tracking, the RX prematurely forwards flit #2 to the upper layer, which then processes request C. Later, upon receiving flit #3, the RX detects the missing flit and initiates a retry starting from flit #1.

However, since request C has already been processed, the retransmitted flit leads to duplicated execution of request C. It is important to note that duplicate flit detection is typically confined to the link layer, and once a duplicate escapes this layer, there is no further protection at higher layers to detect or mitigate its effects. As a result, the transmitter receives four data—A, C, B, and C—despite having issued only three original requests. This inconsistency may lead to FIFO overflows, buffer corruption, and misalignment between requests and responses at the transaction layer.

Fig. 5b presents another failure scenario involving data-carrying flits. CXL supports out-of-order delivery for transactions with different *Command Queue IDs (CQIDs)*; that is, data associated with distinct CQIDs may arrive in any order without violating protocol correctness. However, when multiple transactions share the same CQID, CXL requires in-order delivery of data to ensure that data is correctly reconstructed at the receiver.



(a) A failure due to duplicate requests.



(b) A failure due to out-of-order data.

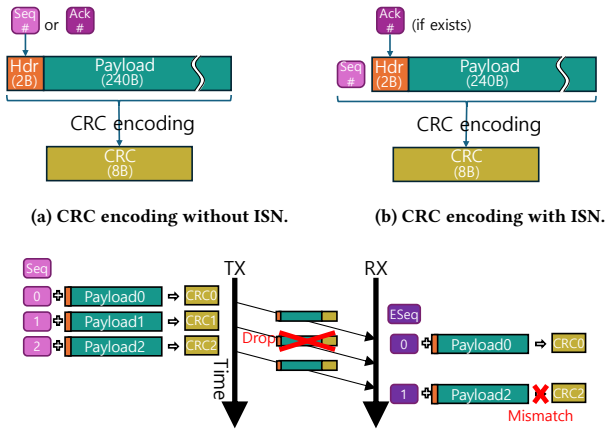
Figure 5: Examples of CXL's failure at the transaction layer.

In this example, data flits B and C are part of the same CQID and must be delivered sequentially. Due to the absence of proper sequence enforcement, the RX erroneously forwards flit C ahead of flit B, resulting in out-of-order delivery within the same queue. This violation may cause the application layer to process incorrect or misaligned data, compromising functional correctness.

5 IMPLICIT SEQUENCE NUMBER

The previous section underscores the challenges of achieving both high reliability and high efficiency in CXL and other modern interconnect protocols. To address these conflicting requirements, this paper introduces the concept of *Implicit Sequence Number (ISN)*, which enables accurate flit sequence tracking without explicitly transferring sequence numbers. It eliminates the need for additional header fields, maintaining high bandwidth efficiency while ensuring reliable and ordered data transmission.

The core idea of ISN is to integrate sequence numbers directly into the CRC encoding and decoding process. Similar to the baseline CXL protocol, both the sender and receiver maintain local sequence counters: SeqNum for the sender and ESeqNum for the receiver. In the baseline CXL protocol, the sender embeds either SeqNum or AckNum in the header and generates an 8-byte CRC checksum over the 2-byte header and 240-byte payload (Fig. 6a). As a result,



(a) CRC encoding without ISN.

(b) CRC encoding with ISN.

(c) An example of flit drop detection using ISN.

Figure 6: An overview of Implicit Sequence Number (ISN).

only one number—either SeqNum or AckNum—can be transmitted to the opposite side.

In contrast, ISN uses the header field to carry only the AckNum. In cases where no acknowledgment is required (i.e., a non-piggybacking flit), this field can be filled with zeros. Meanwhile, the SeqNum information is not explicitly transferred within the flit, but the sender generates the CRC checksum over the header, payload, and the SeqNum (Fig. 6b). The encoding process uses the same CRC polynomial over the wider input, incorporating the sequence number to provide an additional layer of sequence validation.

Figure 6c presents an example of flit drop detection using ISN. After encoding, only the payload and CRC are transmitted; the SeqNum itself is not included in the flit. The sender then increments its local SeqNum in preparation for the next transmission. On the receiver side, the CRC is re-generated using the received 240-byte payload and the receiver's 10-bit local ESeqNum. If the re-generated CRC matches the received CRC, it indicates that: (1) the payload has not been altered, and (2) the sequence numbers align correctly. In this case, the receiver forwards the flit and increments ESeqNum for the next flit.

In the case of a dropped flit, such as $flit_N$, the following flit ($flit_{N+1}$) will have a CRC generated by the sender using sequence number $(N+1)$. The receiver, however, will attempt to decode it with the expected sequence number N in its local counter, resulting in a CRC mismatch. This mismatch signals a flit corruption or drop, enabling the receiver to detect and handle the error accordingly. This implicit sequence tracking ensures that any missing or out-of-order flits are quickly identified through CRC validation, thereby preserving sequence continuity without explicit sequence number transmission.

A limitation of this approach is that it verifies sequence integrity only in a binary manner (i.e., pass or fail) and does not support reordering, as it lacks explicit sequence numbers. In conventional software-centric protocols like TCP/IP, explicit sequence numbers enable the receiver to reorder out-of-order packets and ensure in-order delivery. This is typically achieved by buffering packets in system memory and allowing the operating system to reconstruct the correct sequence before delivering the data to the application [1]. Such reassembly supports selective repeat—where only the missing packets are retried—and enables multi-path routing, allowing packets from the same flow to traverse different network paths to avoid congestion.

In contrast, ISN cannot support reordering because it does not transmit sequence numbers explicitly, and CRC mismatches alone cannot distinguish between flit corruption and flit drop. However, reordering is rarely implemented in high-bandwidth chip interconnects such as CXL, primarily due to the substantial on-chip storage overhead it entails. These hardware-facing protocols must deliver data directly to hardware IP blocks, precluding software-based reassembly. Supporting reordering would require dedicated on-chip reassembly buffers at the receiver to temporarily store out-of-order flits until the missing one arrives.

The required size of the reassembly buffer scales with both the link bandwidth and the worst-case skew in flit arrival times. For example, in a multi-path routing scenario with a 1 ms arrival skew, a 16-lane CXL 3.0 link operating at 1 Tbps would require a 1 Gb (128 MB) reassembly buffer. Due to such substantial overheads, the CXL 3.0 specification avoids support for multi-path routing [9, 13, 52].

In selective repeat scenarios, the buffer requirement is smaller but still non-trivial. For instance, if the system incurs a worst-case delay of 1 μs between flit transmission and the transmitter halting further transmission upon receiving a NACK, the receiver must provision a 1 Mb buffer to absorb in-flight flits during that window. While this is far more practical than multi-path buffering—and supported in the CXL specification—the performance benefit may be limited. Strong FEC makes retransmissions rare, and the traffic reduction achieved through selective repeat is often marginal compared to the associated buffering cost. As a result, many high-speed interconnects—including PCIe and CXL—favor simpler go-back-N schemes over selective repeat. In this context, ISN represents a practical and justifiable design trade-off, aligning with the cost, complexity, and performance constraints of modern chip interconnects.

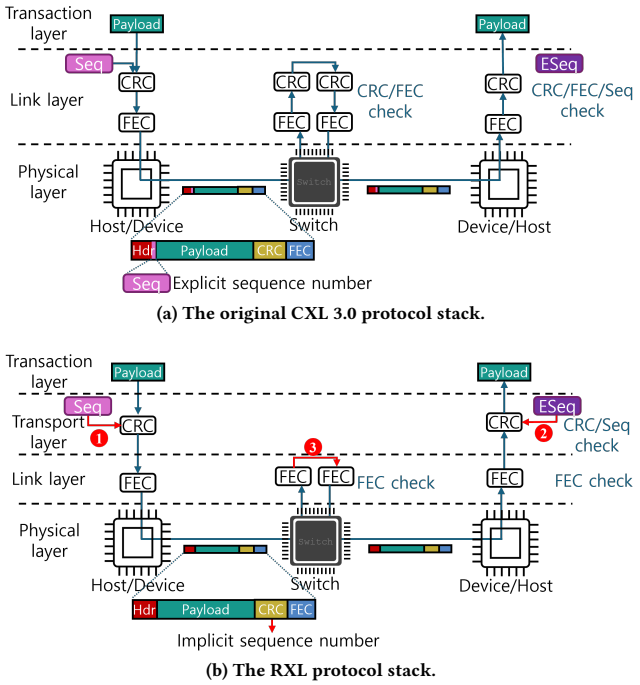
6 RXL: AN EXTENSION OF CXL WITH ISN

Though conceptually straightforward, implementing ISN in chip interconnects like CXL requires careful consideration. This section introduces RXL, an extension of the CXL protocol, to deliver end-to-end protection against flit errors with minimal overhead.

6.1 Overview

The main challenge in implementing ISN lies in its reliance on sequence numbers for CRC decoding. This requires switching devices to maintain state and track sequence numbers of all connections passing through the switch. However, most switching devices are stateless, meaning they do not retain information about requests or responses after forwarding them. Consequently, implementing ISN in a straightforward manner may introduce additional overheads to switches, including connection state tracking and storage for sequence numbers.

To address this, RXL leverages the existing capabilities of FEC at the link layer. While FEC is primarily designed for error correction, it can also detect most uncorrectable errors (Section 2.5). Although FEC's error detection capability is not exhaustive, it provides early



(a) The original CXL 3.0 protocol stack.

(b) The RXL protocol stack.

Figure 7: A comparison of CXL and RXL protocol stacks.

identification of uncorrectable errors, reducing unnecessary forwarding of erroneous flits. Any erroneous flits that bypass FEC's detection are comprehensively validated at the endpoint using a robust 64-bit CRC, ensuring an exceedingly low probability of undetected data corruption.

Therefore, RXL restructures the CXL protocol stack, which employs CRC and FEC at the link layer. In the revised stack (Fig. 7), FEC remains at the link layer, correcting the majority of errors and detecting most of the uncorrectable errors. Meanwhile, CRC functionality is elevated to the transport layer, where it operates as ECRC between the endpoints. This ECRC ensures in-order delivery of flits using ISN and provides highly robust detection of flit corruption using a 64-bit CRC, enhancing both reliability and scalability.

6.2 Flit Structure

The flit structure in RXL retains the original format: a 2B header with FSN and ReplayCmd fields, an 8B CRC, and a 6B FEC. The FSN field is filled with zeros in non-piggybacking scenarios (ReplayCmd = 0) and carries AckNum in piggybacking scenarios (ReplayCmd ≠ 0). Meanwhile, RXL ensures sequence integrity by embedding the sequence number into the CRC, highlighted as 1 in Fig. 7b. This mechanism validates sequence continuity without relying on explicit sequence numbers in the header.

To further optimize bandwidth, an alternative flit structure may omit the FSN and ReplayCmd fields entirely, reallocating their 12 bits to expand the payload or other control fields. In such configurations, ACK and NAK information could be conveyed as standalone packed messages rather than being embedded in the flit header (i.e., no ACK piggybacking), improving data transfer efficiency while simplifying header design.

6.3 Endpoints

In RXL, the originator incorporates the sequence number (SeqNum) into the CRC generation process, ensuring that the CRC encodes both the payload and its sequence information. This CRC, combined with the payload, is then processed through FEC using the same mechanism as the original CXL protocol. At the final destination, the FEC decoder corrects errors in the payload and CRC. The receiver validates the decoded payload and CRC against its local ESeqNum, as shown with 2 in Fig. 7b.

Notably, this CRC validation now resides at the transport layer (i.e., ECRC) and is performed exclusively by the endpoints. This ensures sequence continuity validation while simplifying the switches, which no longer need to track sequence numbers. Another advantage of this transport-layer ECRC is its ability to detect errors occurring within switching devices, such as buffer corruption or switching logic errors. While link-layer CRC and FEC can protect flits during transmission across links, they do not cover errors introduced during internal operations within switches. As a result, switching devices often rely on parity or ECC for internal error protection; however, these mechanisms may leave gaps, such as undetected multi-bit errors or failures in parity protection logic. RXL complements these measures by ensuring that any corruption introduced within the switch is identified by the strong ECRC at the endpoint, providing robust end-to-end data integrity.

6.4 Switches

Switches in RXL operate at the link layer, handling errors through FEC only (See 3 in Fig. 7b). They decode incoming flits using FEC, correcting errors up to 3-byte bursts with 3-way interleaved SSC, and re-encode the flits with FEC before forwarding them to the next node. This link-layer FEC effectively corrects most errors and detects the majority of longer burst errors.

In cases where an error is detected but uncorrectable, the switch discards the flit and reports the issue to the originator for diagnostic purposes, as in the original protocol. By limiting error handling to FEC and avoiding sequence tracking, RXL simplifies switch operations, reducing overhead and complexity while maintaining reliable data forwarding.

7 EVALUATION

This section evaluates the reliability and performance impacts of RXL compared to the existing CXL protocol, alongside an analysis of its hardware overhead.

7.1 Reliability

We define a protocol failure as 1) a flit with corrupted data being forwarded to the application layer ($Fail_{data}$), or 2) a flit being forwarded to the application layer in an incorrect order ($Fail_{order}$). The application-level impact of these failures depends on the flit's contents and application behavior, which are beyond the scope of this evaluation. Instead, this analysis focuses on assessing the occurrence rates of these protocol failures. The evaluation begins

with configurations involving direct connections and extends to configurations involving multi-level switching.

7.1.1 CXL in Direct Connections. In this configuration, the host and device are directly connected via a single CXL link, and we assume that all flits arrive at the endpoint. We calculate the *Flit Error Rate (FER)* based on independent bit errors. Assuming a BER of 10^{-6} and a flit_size of 2048 bits (256 bytes) in CXL 3.0:

$$FER = 1 - (1 - BER)^{flit_size} \simeq 2.0 \times 10^{-3} \quad (1)$$

This means approximately 0.2% of flits are erroneous, equating to 1 million erroneous flits out of 500 million flits per second on a ×16 CXL link.

Next, we assess the probabilities of these erroneous flits being uncorrectable and detectable by FEC and CRC. The lower bound for uncorrectable flit error rate after FEC is provided by the PCIe 6.0 standard [34]:

$$FER_{UC} = 3.0 \times 10^{-5} \quad (2)$$

From this, we can expect FEC to correct more than 98.5% of erroneous flits, as derived from:

$$p_{correct} = 1 - \frac{FER_{UC}}{FER} \quad (3)$$

Among the remaining 1.5% uncorrectable flits, the 64-bit CRC detects the vast majority with an exceedingly low undetected error probability of 2^{-64}. The resulting undetectable flit error rate is

$$FER_{UD} = FER_{UC} \times 2^{-64} \simeq 1.6 \times 10^{-24} \quad (4)$$

Note that this FER_{UD} represents an upper bound, as burst errors no longer than 64 bits are detected with 100% certainty.

Given a ×16 CXL device transferring 500M flits per second, this undetected error rate translates to a *Failure In Time* (FIT, number of failures expected over one billion hours) of:

$$FIT_{device} = FER_{UD} \times 500,000,000 \times 3,600 \times 10^9 \simeq 2.9 \times 10^{-3} \quad (5)$$

This exceptionally low FIT value demonstrates that the direct connection configuration provides extremely high reliability, surpassing the typical target for server-grade devices, which have FIT values in the range of a few hundred [51].

7.1.2 CXL in Single-level Switched Environments. In this configuration, the host and device communicate through a single-level switch, with data transmitted over two links. The switch discards uncorrectable flits detected on the first link between the originator and the switch. As a result, the rate of dropped flits at the endpoint is the same as the rate of uncorrectable flits at the first link:

$$FER_{drop} = FER_{UC} = 3.0 \times 10^{-5} \quad (6)$$

Dropped flits can lead to ordering failures when the subsequent flit conveys an AckNum instead of a SeqNum. The probability of this occurring depends on the ACK coalescing level, denoted as $p_{coalescing}$. For example, $p_{coalescing} = 0.1$ indicates one in ten flits carries an AckNum. The ordering failure rate (with $p_{coalescing} = 0.1$) is thus given by:

$$FER_{order} = FER_{drop} \times p_{coalecing} = 3.0 \times 10^{-6} \quad (7)$$

This rate far exceeds the undetected data failure rate after CRC and is used as the primary consideration for simplicity. For a ×16 device transmitting 500M flits per second, the resulting FIT is:

$$FIT_{device} = FER_{order} \times 500,000,000 \times 3,600 \times 10^9 \simeq 5.4 \times 10^{15} \quad (8)$$

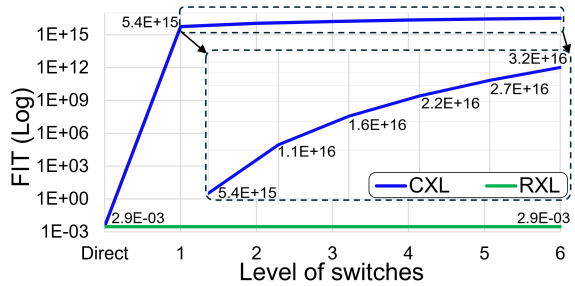


Figure 8: Comparison of FIT_{device} of CXL and RXL against increasing switching levels.

This extraordinarily high FIT value of ordering failures highlights the vulnerability of the current CXL protocol in switched environments. The strong detection and correction mechanisms of CRC and FEC, while effective for data integrity, do not address this ordering failure mode, as they operate exclusively on received flits and do not mitigate flit drops. In CXL, flit drops are detected exclusively through explicit sequence numbers. However, the use of ACK piggybacking can prevent some flits from carrying their sequence numbers, leaving these flits unprotected against drops and resulting in undetected ordering violations.

7.1.3 RXL in Single-level Switched Environments. This analysis considers the same interconnect topology as before (single-level switching) but upgrades the protocol to RXL. Unlike the original CXL protocol, RXL integrates ISN, allowing it to detect all flit drops and prevent ordering failures ($Fail_{order}$) through retries. Consequently, the only remaining failure type is corrupted data being forwarded to the application layer ($Fail_{data}$).

Flits dropped on the first link are eventually identified by the endpoint with ISN and retried. The probability of data failures at the endpoint, after accounting for retries, is given by:

$$FER_{UD} = (1 + FER_{UC}) \times 2^{-64} \simeq 1.6 \times 10^{-24} \quad (9)$$

This extremely low probability is driven by the 64-bit CRC's strong error detection capability.

The corresponding FIT rate for a ×16 device is:

$$FIT_{device} = 1.6 \times 10^{-24} \times 500,000,000 \times 3,600 \times 10^9 \simeq 2.9 \times 10^{-3} \quad (10)$$

This reduction in FIT value, exceeding 10^{18} times lower than that of the original CXL protocol, demonstrates the dramatic reliability improvements achievable by RXL. By eliminating ordering failures and ensuring robust end-to-end error detection, RXL provides an efficient and reliable solution for scaling out chip interconnect networks in multi-node systems.

7.1.4 Multi-level Switched Environments. The dropped flits and ordering failures in CXL pose significant challenges to scaling out systems, especially in multi-level switched environments. Fig. 8 compares the FIT_{device} of CXL and RXL as the level of switching increases. Both protocols exhibit high reliability in direct connection configurations. However, with the introduction of single-level switching, the reliability of CXL declines drastically, decreasing

by approximately 10^{18} times due to its susceptibility to flit drops and sequence violations. Furthermore, these failure rates apply to a single device; in multi-node systems with numerous parallel connections, the compounded effect makes the current CXL protocol insufficient for maintaining reliable chip interconnect networks.

In contrast, the reliability of RXL remains nearly unchanged, even with increasing levels of switching. Its embedded sequence validation mechanism ensures robust error detection and ordering integrity, effectively mitigating vulnerabilities that undermine CXL in switched environments. This analysis underscores the importance of incorporating advanced sequence tracking mechanisms like RXL to maintain reliability and efficiency in switched and large-scale interconnect environments.

7.2 Performance

The performance impact of the protection schemes is evaluated by estimating the bandwidth loss they introduce. This analysis considers a ×16 link transmitting 256B flits every 2ns. A go-back-N retry scheme with a 100ns latency is assumed [42], during which the link is occupied with retransmitted flits.

7.2.1 CXL in Direct Connections. Bandwidth loss due to retries is calculated by comparing the effective duration of flit transmission with and without retries. Flits without a retry, occurring with a probability of $(1 - FER_{UC})$, occupy the channel for 2ns each. Conversely, flits requiring retries, occurring with a probability of FER_{UC}, occupy the channel for 102ns. The bandwidth loss can thus be estimated as:

$$BW_{loss} = 1 - \frac{2ns}{(1 - FER_{UC}) \times 2ns + FER_{UC} \times 102ns} \simeq 0.0015 \quad (11)$$

This calculation shows that retries result in a bandwidth loss of approximately 0.15%, demonstrating that CXL ensures reliable and efficient communication even under retry scenarios in direct connections. Moreover, adopting a more efficient retry scheme, such as selective retry, could further reduce this loss, enhancing overall link utilization and system performance.

7.2.2 CXL in Switched Environments. In this analysis, we explore two options for CXL to handle flit drops. The first option continues to use ACK piggybacking, minimizing performance degradation by avoiding the overhead of separate ACK flits. However, this approach is vulnerable to frequent protocol failures caused by flit drops.

In this scenario, flits dropped at the first link are eventually detected by the explicit sequence number of the immediately following or subsequent flits at the endpoint, initiating retries. The rate of such dropped flits at the first link is equal to FER_{UC}. Similarly, uncorrectable flit errors at the second link, also occurring at a rate of FER_{UC}, initiate additional retries. Consequently, the cumulative rate of flits requiring retries is $2 \times FER_{UC}$. The bandwidth loss from these retries can be estimated similarly to Eqn. 11:

$$BW_{loss} = 1 - \frac{2ns}{(1 - 2 \times FER_{UC}) \times 2ns + 2 \times FER_{UC} \times 102ns} \simeq 0.0030 \quad (12)$$

This calculation indicates that the first option results in a small bandwidth loss of approximately 0.3%. However, this approach suffers from a critical reliability issue related to flit drops, as it utilizes piggybacked ACKs and cannot detect some flit ordering failures.

The second option disables ACK piggybacking and instead uses separate flits for ACK and NACK messages. This approach ensures precise sequence tracking by embedding explicit sequence numbers in each flit. However, it incurs a significant bandwidth cost due to the additional overhead of separate ACK flits.

The bandwidth loss from these ACK flits can be estimated as:

$$BW_{loss} = p_{coalescing} \quad (13)$$

Without ACK coalescing ($p_{coalescing} = 1$), the bandwidth loss could reach 100% as the link is fully utilized to transfer ACK flits in the reverse direction. To mitigate this overhead, coalescing more ACK messages into a single ACK flit (reducing $p_{coalescing}$) can be employed. However, achieving higher levels of ACK coalescing requires larger buffers to store transmitted flits until the corresponding ACK is received. This increased buffering demand adds system complexity and resource requirements, and it also imposes practical limits on the coalescing level.

7.2.3 RXL in Switched Environments. With RXL, all flits dropped at the first link, occurring at a rate of FER_{UC}, are immediately detected by their subsequent flits through embedded sequence tracking, initiating a retry. Similar to the CXL approach, almost all uncorrectable flit errors at the second link are detected by the ECRC and trigger a retry. As a result, the rate of retries is nearly identical to that calculated in Eqn. 12:

$$BW_{loss} = 1 - \frac{2ns}{(1 - 2 \times FER_{UC}) \times 2ns + 2 \times FER_{UC} \times 102ns} \simeq 0.0030 \quad (14)$$

This result demonstrates that RXL incurs similar bandwidth loss (approximately 0.3%) as the CXL approach with ACK piggybacking while providing enhanced reliability by embedding sequence validation directly into the ECRC.

This evaluation highlights that RXL ensures robust data and sequence integrity without adversely affecting performance, even in multi-level switching environments. By seamlessly integrating sequence validation into the error-checking mechanism, RXL addresses the shortcomings of CXL, such as vulnerability to sequence violations. As a result, RXL provides a scalable and efficient solution tailored to the demands of modern high-speed interconnects, offering enhanced reliability while maintaining high performance.

7.3 Hardware Overheads

Although ISN integrates SeqNum into CRC encoding and decoding, it introduces minimal hardware overhead. In the encoding process, the 10-bit SeqNum is XORed with the lower 10 bits of the 240B payload. The XORed result then feeds into the existing CRC encoder to generate the checksum according to the new scheme, adding only 10 parallel XOR gates. Similarly, in CRC decoding, the ESeqNum is XORed with the received payload before entering the existing decoder. As a result, both the encoder and decoder require only 10 XOR gates, with an increase of just one additional logic depth. Additionally, ISN eliminates the need for the 10-bit comparator that previously compared SeqNum and ESeqNum. Consequently, the overall hardware overhead from ISN is minimal, requiring only a few additional gates.

8 CONCLUSION

This paper presents ISN, a novel concept to achieve reliable and efficient chip interconnect, and RXL, a scalable and reliable extension of the CXL protocol. By embedding sequence tracking into CRC, ISN eliminates the need for explicit sequence numbers in flit headers, reducing header overhead without compromising sequence integrity. The proposed RXL enhances the robustness of the chip interconnect by ensuring end-to-end data and sequence validation while maintaining compatibility with existing flit structures.

Our evaluation demonstrates that RXL achieves the same level of performance as the existing method while addressing the critical reliability vulnerability of ordering failures. Additionally, RXL delivers strong end-to-end protection against data corruption, ensuring even errors internal to switches are effectively detected and mitigated by 64-bit CRC. These features make RXL a highly effective solution for modern high-speed, error-prone interconnect environments.

The lightweight nature of ISN and its seamless integration with existing protocols position it as a scalable and efficient approach for chip interconnects. Future work could explore extending ISN to other protocols and systems, such as Network-on-Chip (NoC) and chiplet interconnects, or adapting its principles to address challenges in broader communication protocols, further enhancing reliability across diverse computing architectures.

REFERENCES

[1] 1982. IP datagram reassembly algorithms. RFC 815. https://doi.org/10.17487/RFC0815

[2] 2022. IEEE Standard for Ethernet. *IEEE Std 802.3* (2022), 1–7025. https://doi.org/10.1109/IEEESTD.2022.9844436

[3] Dennis Abts and Cray Red Storm. 2011. Cray XT4 and Seastar 3-D Torus Interconnect. *Encyclopedia of Parallel Computing* 6 (2011).

[4] Mohamed Ahmed, Ahmed Abdelaziz, and Tawfiq Musah. 2024. Multi-Stage Speculative PAM4 Decision Feedback Equalizer for Short Reach Applications. *IEEE Transactions on Circuits and Systems I: Regular Papers* (2024), 1–13. https://doi.org/10.1109/TCSI.2024.3391657

[5] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 1–15. https://doi.org/10.1109/SC41404.2022.00051

[6] Fault-Tolerant Computing. 1986. Fault-Tolerant Computing: Theory and Techniques. *Englewood Cliffs, NJ* 2 (1986).

[7] CXL Consortium. 2019. *Computer Express Link 1.1 specification.* https://computeexpresslink.org/cxl-specification/

[8] CXL Consortium. 2020. *Computer Express Link 2.0 specification.* https://computeexpresslink.org/cxl-specification/

[9] CXL Consortium. 2023. *Computer Express Link 3.1 specification.* https://computeexpresslink.org/cxl-specification/

[10] Ultra Accelerator Link Consortium. 2025. *Ultra Accelerator Link Specification.* https://ualinkconsortium.org/specification/

[11] Shengkun Cui, Archit Patke, Ziheng Chen, Aditya Ranjan, Hung Nguyen, Phuong Cao, Saurabh Jha, Brett Bode, Gregory Bauer, Chandra Narayanaswami, et al. 2025. Characterizing GPU Resilience and Impact on AI/HPC Systems. *arXiv preprint arXiv:2503.11901* (2025).

[12] John Danskin and Denis Foley. 2016. Pascal GPU with NVLink. In *2016 IEEE Hot Chips 28 Symposium (HCS)*. IEEE Computer Society, 1–24.

[13] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. 2024. An Introduction to the Compute Express Link (CXL) Interconnect. *ACM Comput. Surv.* 56, 11, Article 290 (2024), 37 pages. https://doi.org/10.1145/3669900

[14] Sheng Di, Hanqi Guo, Rinku Gupta, Eric R. Pershey, Marc Snir, and Franck Cappello. 2019. Exploring Properties and Correlations of Fatal Events in a Large-Scale HPC System. *IEEE Transactions on Parallel and Distributed Systems* 30, 2 (2019), 361–374. https://doi.org/10.1109/TPDS.2018.2864184

[15] DIODES. 2022. *DIODES PI7C9X3G816GP PCIe switch.* https://www.diodes.com/part/view/PI7C9X3G816GP

[16] Abhimanyu Dubey and 538 number of other authors. 2024. The Llama 3 Herd of Models. *arXiv e-prints*, Article arXiv:2407.21783 (July 2024), arXiv:2407.21783 pages. https://doi.org/10.48550/arXiv.2407.21783 arXiv:2407.21783 [cs.AI]

[17] Saurabh Gupta, Tirthak Patel, Christian Engelmann, and Devesh Tiwari. 2017. Failures in large scale systems: long-term measurement, analysis, and implications. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17)*. Association for Computing Machinery, New York, NY, USA, Article 44, 12 pages. https://doi.org/10.1145/3126908.3126937

[18] Intel. 2022. *Ethernet Controller X710/ XXV710/XL710 Datasheet.* https://cdrdv2-public.intel.com/332464/332464_710_Series_Datasheet_v_4_1.pdf

[19] Intel Corp. 2019. PAM4 Signaling Fundamentals.

[20] Giyong Jung, Hee Ju Na, Sang-Hyo Kim, and Jungrae Kim. 2024. Dual-Axis ECC: Vertical and Horizontal Error Correction for Storage and Transfer Errors. In *2024 IEEE 42nd International Conference on Computer Design (ICCD)*. 409–417. https://doi.org/10.1109/ICCD63220.2024.00069

[21] Dongwhee Kim, Jaeyoon Lee, Wonyeong Jung, Michael Sullivan, and Jungrae Kim. 2023. Unity ECC: Unified Memory Protection Against Bit and Chip Errors. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–16.

[22] Jungrae Kim, Michael Sullivan, Sangkug Lym, and Mattan Erez. 2016. All-Inclusive ECC: Thorough End-to-End Protection for Reliable Computer Memory. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 622–633. https://doi.org/10.1109/ISCA.2016.60

[23] Hyokeun Lee, Kwanseok Choi, Hvuk-Jae Lee, and Jaewoong Sim. 2023. SDM: Sharing-Enabled Disaggregated Memory System with Cache Coherent Compute Express Link. In *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 86–98.

[24] Chao Li, Yang Hu, Ruijin Zhou, Ming Liu, Longjun Liu, Jingling Yuan, and Tao Li. 2013. Enabling datacenter servers to scale out economically and sustainably. In *Proceedings of the 46th annual IEEE/ACM international symposium on microarchitecture*. 322–333.

[25] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. 2023. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 574–587.

[26] Yujin Lim, Dongwhee Kim, and Jungrae Kim. 2024. SELCC: Enhancing MLC Reliability and Endurance with Single-Cell Error Correction Codes. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1–6. https://doi.org/10.23919/DATE58400.2024.10546752

[27] S Lin and DJ Costello. 2004. Error control coding: fundamentals and applications.

[28] Pejman Lotfi-Kamran, Boris Grot, Michael Ferdman, Stavros Volos, Onur Kocberber, Javier Picorel, Almutaz Adileh, Djordje Jevdjic, Sachin Idgunji, Emre Ozer, et al. 2012. Scale-out processors. *ACM SIGARCH Computer Architecture News* 40, 3 (2012), 500–511.

[29] MARVELL. 2008. *Link Street 88E6060 - Unrestricted.* https://www.insidegadgets.com/wp-content/uploads/2014/07/88E6060.pdf

[30] Steve McDowell. 2024. *UALink Alliance for Accelerator Interconnects.* Technical Report. NAND Research. https://nand-research.com/wp-content/uploads/2024/05/2024-05-30-RN-UALink-1.pdf Accessed: November 20, 2024.

[31] Rajan Narasimha, Nirmal Warke, and Naresh Shanbhag. 2009. Impact of DFE Error Propagation on FEC-Based High-Speed I/O Links. In *GLOBECOM 2009 - 2009 IEEE Global Telecommunications Conference*. 1–6. https://doi.org/10.1109/GLOCOM.2009.5426027

[32] NVIDIA. 2024. *NVLink and NVLink Switch.* https://www.nvidia.com/en-us/datacenter/nvlink/

[33] Dylan Patel and Aleksandar Kostovic. 2023. *TPUv5e: The New Benchmark in Cost-Efficient Inference and Training for <200B Parameter Models.* https://semianalysis.com/2023/09/01/tpuv5e-the-new-benchmark-in-cost/

[34] pcisig. 2024. *PCI Express 6.0 Specification.* https://pcisig.com/pci-express-6.0-specification

[35] Hongwu Peng, Caiwen Ding, Tong Geng, Sutanay Choudhury, Kevin Barker, and Ang Li. 2024. Evaluating Emerging AI/ML Accelerators: IPU, RDU, and NVIDIA/AMD GPUs. In *Companion of the 15th ACM/SPEC International Conference on Performance Engineering* (London, United Kingdom) *(ICPE '24 Companion)*. Association for Computing Machinery, New York, NY, USA, 14–20. https://doi.org/10.1145/3629527.3651428

[36] PLX Technology. 2008. *BroadCOM PEX 8613 PCIe switch.* https://docs.broadcom.com/doc/pcie-errorrecovery-whitepaper

[37] Zhiyun Qian and Z. Morley Mao. 2012. Off-path TCP Sequence Number Inference Attack - How Firewall Middleboxes Reduce Security. In *2012 IEEE Symposium on Security and Privacy*. 347–361. https://doi.org/10.1109/SP.2012.29

[38] RENESAS. 2007. *RENESAS IDT 89HPES8T5 PCIe switch user manual.* https://www.renesas.com/en/document/mah/pes8t5-device-usermanualpdf

[39] Sushant Rewaskar, Jasleen Kaur, and Don Smith. 2006. Empirical Analysis of TCP Losses and Its Detection/Recovery Mechanisms. (2006).

[40] Yiannakis Sazeides, Emre Özer, Danny Kershaw, Panagiota Nikolaou, Marios Kleanthous, and Jaume Abella. 2013. Implicit-storing and redundant-encoding-of-attribute information in error-correction-codes. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture* (Davis, California) *(MICRO-46)*. Association for Computing Machinery, New York, NY, USA, 160–171. https://doi.org/10.1145/2540708.2540723

[41] Aakash Sharma, Vivek M. Bhasi, Sonali Singh, George Kesidis, Mahmut T. Kandemir, and Chita R. Das. 2024. GPU Cluster Scheduling for Network-Sensitive Deep Learning. arXiv:2401.16492 [cs.PF] https://arxiv.org/abs/2401.16492

[42] Debendra Das Sharma. 2020. PCI Express 6.0 specification: A low-latency, high-bandwidth, high-reliability, and cost-effective interconnect with 64.0 GT/s PAM-4 signaling. *IEEE Micro* 41, 1 (2020), 23–29.

[43] Debendra Das Sharma. 2020. PCI Express® 6.0 Specification at 64.0 GT/s with PAM-4 signaling: a low latency, high bandwidth, high reliability and cost-effective interconnect. In *2020 IEEE Symposium on High-Performance Interconnects (HOTI)*. 1–8. https://doi.org/10.1109/HOTI51249.2020.00016

[44] Debendra Das Sharma. 2023. Compute Express Link (CXL): Enabling Heterogeneous Data-Centric Computing With Heterogeneous Memory Hierarchy. *IEEE Micro* 43, 2 (2023), 99–109. https://doi.org/10.1109/MM.2022.3228561

[45] Debendra Das Sharma and Swadesh Choudhary. 2024. Pipelined and Partitionable Forward Error Correction and Cyclic Redundancy Check Circuitry Implementation for PCI Express 6.0 and Compute Express Link 3.0. *IEEE Micro* 44, 2 (2024), 50–59. https://doi.org/10.1109/MM.2023.3328832

[46] Yuseok Song, Sangjae Park, Michael B. Sullivan, and Jungrae Kim. 2022. SEC-BADAEC: An Efficient ECC With No Vacancy for Strong Memory Protection. *IEEE Access* 10 (2022), 89769–89780. https://doi.org/10.1109/ACCESS.2022.3201525

[47] Song, Chihun and Kim, Michael Jaemin and Wang, Tianchen and Ji, Houxiang and Huang, Jinghan and Jeong, Ipoom and Park, Jaehyun and Nam, Hwayong and Wi, Minbok and Ahn, Jung Ho and others. 2024. TAROT: A CXL SmartNIC-Based Defense Against Multi-bit Errors by Row-Hammer Attacks. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 981–998.

[48] Michael B. Sullivan, Mohamed Tarek Ibn Ziad, Aamer Jaleel, and Stephen W. Keckler. 2023. Implicit Memory Tagging: No-Overhead Memory Safety Using Alias-Free Tagged ECC. In *International Symposium on Computer Architecture* (Orlando, FL, USA) *(ISCA '23)*. Association for Computing Machinery, New York, NY, USA, Article 67, 13 pages. https://doi.org/10.1145/3579371.3589102

[49] Andrew S. Tanenbaum, Nick Feamster, and David J. Wetherall. 2021. *Computer Networks* (6th ed.). Pearson, Boston.

[50] Universal Chiplet Interconnect Express Consortium. 2025. *Universal Chiplet Interconnect Express*. https://www.uciexpress.org/specifications

[51] Fan Wang and Vishwani D. Agrawal. 2010. Soft error considerations for computer web servers. In *2010 42nd Southeastern Symposium on System Theory (SSST)*. 269–274. https://doi.org/10.1109/SSST.2010.5442820

[52] Shu-Ting Wang and Weitao Wang. 2023. Aurelia: CXL Fabric with Tentacle. In *Proceedings of the 4th Workshop on Resource Disaggregation and Serverless* (Koblenz, Germany) *(WORDS '23)*. Association for Computing Machinery, New York, NY, USA, 29–36. https://doi.org/10.1145/3605181.3626287

[53] Kang Xi, Yu-Hsiang Kao, and H Jonathan Chao. 2013. A petabit bufferless optical switch for data center networks. *Optical interconnects for future data center networks* (2013), 135–154.